

Fetching and Visualizing Official Statistics with R

Nicolas
Eesti Pank

2025-04-17

Table of contents

- Interfaces to Official Statistics
 - How to fetch data (from DBnomics using R)
 - Fetch two (or more) series at once
 - Fetch two series from different datasets of different providers
 - Fetch large amounts of data
 - Accessing Statistics Estonia Data with `pxweb`
 - How to Identify Query Variables

Interfaces to Official Statistics

- Packages or set of classes and methods to read data and metadata documents through exchange frameworks
 - Use **R** (or **Python**) packages to read data from APIs, databases, and web pages
 - Individual packages:
 - **eurostat**: Access data from Eurostat
 - **OECD**: Access data from the OECD API
 - General-purpose packages:
 - **rdbnomics**: Unified access to many economic databases (e.g. ECB, Eurostat, IMF, World Bank)

- Interface standards:
 - **SDMX**: Statistical Data and Metadata Exchange format
 - **pxweb**: Access to data sources using the PX-Web API (e.g. Statistics Sweden, Statistics Estonia)

DBnomics

- **DBnomics** is a database of databases
 - free platform to aggregate publicly-available economic data provided by national and international statistical institutions, but also by researchers and private companies
 - Unified interface to access data from many sources
 - Harmonized data formats and metadata
 - Data series are available upon release by the provider
 - Each revision is archived to build a real-time database

How to fetch data (from DBnomics using R)

- DBnomics R client

```
1 install.packages("rdbnomics")  
2 library(rdbnomics)
```

Packages used in this tutorial

-  Fetching data ([rdbnomics](#))
-  Data wrangling and transformation ([tidyverse](#))
-  Visualization ([ggplot2](#), [plotly](#))
-  Tabular summaries ([gt](#))
-  Building this presentation ([quarto](#))

```
1 library(quarto) # for compiling Quarto presentations
2 library(rdbnomics) # for accessing economic data via DBnomics
3 library(pxweb)
4 library(tidyverse) # dplyr, ggplot2, readr, etc.
5 library(plotly) # interactive visualizations
6 library(gt) # pretty tables
7 library(countrycode)
```

Example: Fetch Unemployment Data

- Assume we know exactly the series ID we want to fetch
 - Unemployment rate, ILO definition, total, Estonia, from Eurostat

```
1 unemp <- rdb(ids = "Eurostat/ei_lmhr_m/M.PC_ACT.SA.LM-UN-T-TOT.EE") # fetch data
```



```
1 glimpse(unemp)
```

Rows: 296

Columns: 22

```
$ `@frequency`  
$ dataset_code  
$ dataset_name  
$ freq  
$ geo  
$ `Geopolitical entity (reporting)`  
$ indexed_at  
$ indic  
$ Indicator  
$ observations_attributes  
$ original_period  
$ original_value  
$ period  
$ provider_code  
$ s_adj  
$ `Seasonal adjustment`  
$ series_code  
$ series_name
```

```
<chr> "monthly", "monthly", "monthly", "monthly", "mo...  
<chr> "ei_lmhr_m", "ei_lmhr_m", "ei_lmhr_m...  
<chr> "Unemployment rate (%)" - monthly dat...  
<chr> "M", "M", "M", "M", "M", "M", "M", "...  
<chr> "EE", "EE", "EE", "EE", "EE", "EE", "EE", "...  
<chr> "Estonia", "Estonia", "Estonia", "Es...  
<dtm> 2024-10-31 15:26:51, 2024-10-31 15:...  
<chr> "LM-UN-T-TOT", "LM-UN-T-TOT", "LM-UN...  
<chr> "Unemployment according to ILO defin...  
<chr> "OBS_FLAG", "OBS_FLAG", "OBS_FLAG,...  
<chr> "2000-02", "2000-03", "2000-04", "20...  
<chr> "14.9", "14.2", "14.5", "13.9", "14"...  
<date> 2000-02-01, 2000-03-01, 2000-04-01,...  
<chr> "Eurostat", "Eurostat", "Eurostat", "...  
<chr> "SA", "SA", "SA", "SA", "SA", "SA", "...  
<chr> "Seasonally adjusted data, not calen...  
<chr> "M.PC_ACT.SA.LM-UN-T-TOT.EE", "M.PC_...  
<chr> "Monthly - Percentage of population ...
```

```
1 colnames(unemp)
```

```
[1] "@frequency"  
[3] "dataset_name"  
[5] "geo"  
[7] "indexed_at"  
[9] "Indicator"  
[11] "original_period"  
[13] "period"  
[15] "s_adj"  
[17] "series_code"  
[19] "Time frequency"  
[21] "Unit of measure"  
  
"dataset_code"  
"freq"  
"Geopolitical entity (reporting)"  
"indic"  
"observations_attributes"  
"original_value"  
"provider_code"  
"Seasonal adjustment"  
"series_name"  
"unit"  
"value"
```

```
1 # Extract source and series ID from the metadata
2 (source_name <- unique(unemp$dataset_code))
```

```
[1] "ei_lmhr_m"
```

```
1 (provider_code <- unique(unemp$provider_code))
```

```
[1] "Eurostat"
```

```
1 (country_name <- unique(unemp$`Geopolitical entity (reporting)`)) )
```

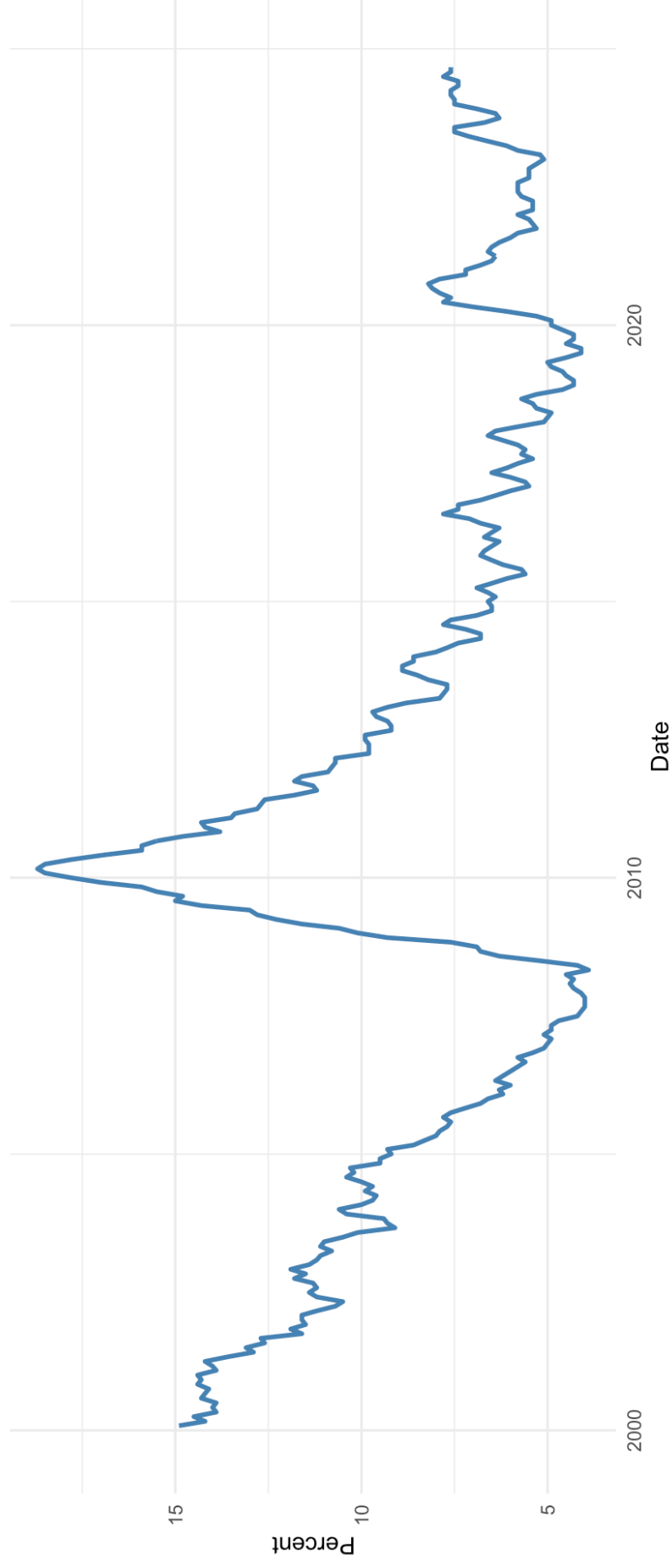
```
[1] "Estonia"
```

```
1 (series_id <- unique(unemp$series_code))
```

```
[1] "M.PC_ACT.SA.LM-UN-T-TOT.EE"
```

```
1 # Plot the data
2 p1 <- ggplot(unemp, aes(x = period, y = value)) +
3   geom_line(color = "steelblue", linewidth = 1) +
4   labs(
5     title = paste("Unemployment Rate in ", country_name),
6     subtitle = paste("Monthly, seasonally adjusted -", provider_code),
7     x = "Date", y = "Percent",
8     caption = paste("Source:", provider_code, " | Dataset:", source_name, " | ID:", series_id)
9   ) +
10  theme_minimal()
11 p1
```

Unemployment Rate in Estonia
Monthly, seasonally adjusted — Eurostat

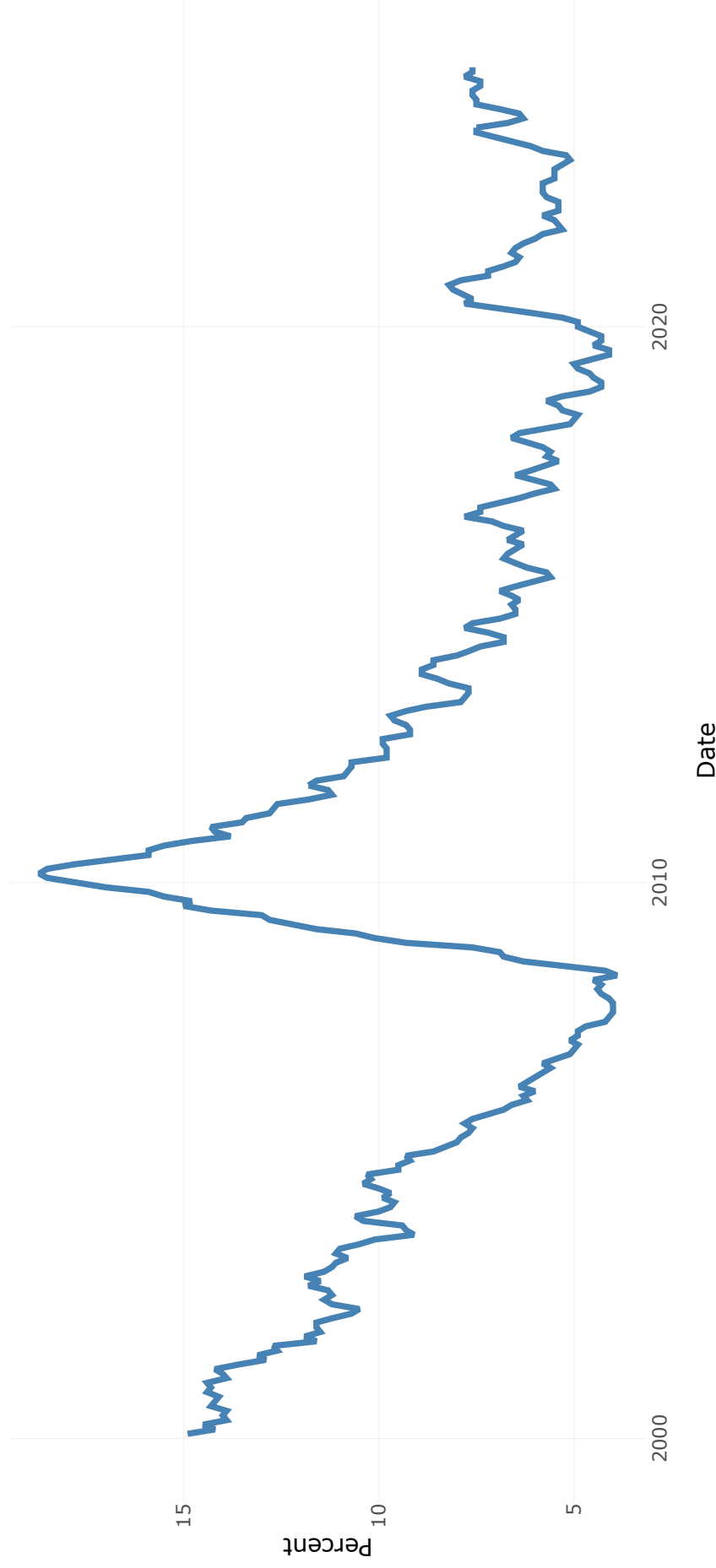


Source: Eurostat | Dataset: ej_lmhr_m | ID: M.PC_ACT.SA.LM-UN-T-TOT.EE

Interactive plot

```
1 ggplotly(p1)
```

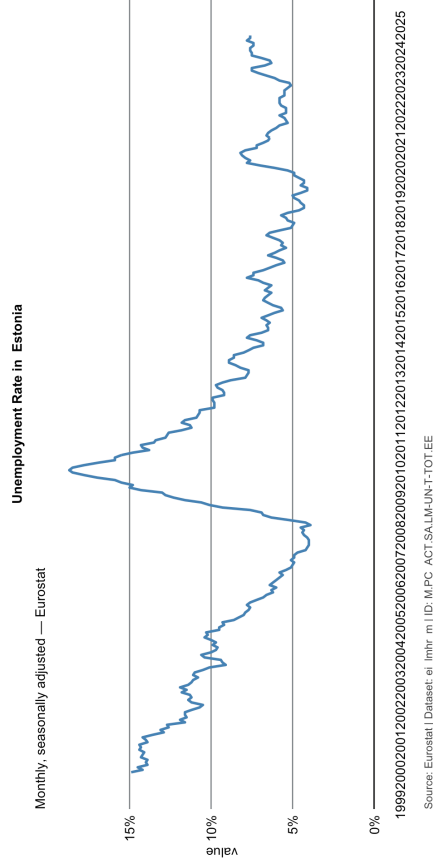
Unemployment Rate in Estonia



```

1 source(here::here("scripts", "theme_ep.R"))
2 ggplot(unemp, aes(x = period, y = value)) +
3   geom_line(color = "steelblue", linewidth = 1) +
4   geom_hline(yintercept = 0, color = "#000", linewidth = 0.5) +
5   scale_x_date(date_breaks = "1 year", date_labels = "%Y") +
6   scale_y_continuous(labels = scales::label_number(suffix = "%")) +
7   labs(
8     title = paste("Unemployment Rate in ", country_name),
9     subtitle = paste("Monthly, seasonally adjusted -", provider_code),
10    x = "",
11    x = "",
12    caption = paste(
13      "Source:",
14      provider_code,
15      "| Dataset:",
16      source_name,
17      "| ID:",
18      series_id
19    )
20  ) +
21  theme_ep()

```



How do we find the series ID/mask/dimensions?

- Go to the [DBnomics website](#)
 - Search directly for a series or pick a provider
 - Search for the [data](#) you want (dataset_code)
 - Click on the [series](#) (series_code)
 - Copy the series ID from the URL

- Show the available datasets of a provider:

```
1 head(rdb_datasets(provider_code = "Eurostat"))
```

```
$Eurostat
      code
      <char>
1: aact_ali01
2: aact_ali02
3: aact_eaa01
4: aact_eaa02
5: aact_eaa03
---
8289: yth_empl_120
8290: yth_empl_130
8291: yth_empl_130
8292: yth_empl_140
8293: yth_empl_140

      name
      <char>
1: Agricultural labour input statistics: absolute figures (1 000 annual work units)
2: Agricultural labour input statistics: indices
3: Economic accounts for agriculture – values at current prices
4: Economic accounts for agriculture – values at n-1 prices
```


- Show the dimensions of a dataset:

```
1 head(rdb_dimensions(provider_code = "Eurostat", dataset_code = "ei_lmhr_m"))
```

```
$Eurostat
$Eurostat$ei_lmhr_m
$Eurostat$ei_lmhr_m$freq
  Time frequency
<char>
1: M Monthly

$Eurostat$ei_lmhr_m$geo
  geo
<char>
1: AT
2: BA
3: BE
4: BG
5: CH
6: CY
7: CZ
8: DE
9: DK
10: EA20
      Euro area – 20 countries (from 2023)

  Geopolitical entity (reporting)
  <char>
1: Austria
2: Bosnia and Herzegovina
3: Belgium
4: Bulgaria
5: Switzerland
6: Cyprus
7: Czechia
8: Germany
9: Denmark
```

- Query to filter/select series from a provider's dataset

```
1 head(rdb_series(  
2   provider = "Eurostat",  
3   dataset_code = "ei_lmhr_m",  
4   query = "United Kingdom"  
5 ))
```

```
$Eurostat  
$Eurostat$ei_lmhr_m      series_code  
      <char>  
1: M.PC_ACT.NSA.LM-UN-F-GT25.UK  
2: M.PC_ACT.NSA.LM-UN-F-LE25.UK  
3: M.PC_ACT.NSA.LM-UN-F-TOT.UK  
4: M.PC_ACT.NSA.LM-UN-M-GT25.UK  
5: M.PC_ACT.NSA.LM-UN-M-LE25.UK  
6: M.PC_ACT.NSA.LM-UN-M-TOT.UK  
7: M.PC_ACT.NSA.LM-UN-T-GT25.UK  
8: M.PC_ACT.NSA.LM-UN-T-LE25.UK  
9: M.PC_ACT.NSA.LM-UN-T-TOT.UK  
10: M.PC_ACT.SA.LM-UN-F-GT25.UK  
11: M.PC_ACT.SA.LM-UN-F-LE25.UK  
12: M.PC_ACT.SA.LM-UN-F-TOT.UK  
13: M.PC_ACT.SA.LM-UN-M-GT25.UK  
14: M.PC_ACT.SA.LM-UN-M-LE25.UK  
15: M.PC_ACT.SA.LM-UN-M-TOT.UK  
16: M.PC_ACT.SA.LM-UN-T-GT25.UK  
.....
```

Fetch two (or more) series at once

- Example: Balance of Payments (BOP) for France and Germany from the IMF for Current Account, Total, Net, Euros, Millions, Annual

Option A

Option B:

Option C:

```
1 # by ID
2 bop <- rdb(ids = c("IMF/BOP/A.FR.BCA_BP6_EUR", "IMF/BOP/A.DE.BCA_BP6_EUR"))
3 bop %>% count(`Reference Area`)
```

	Reference Area	n
1:	France	15
2:	Germany	26

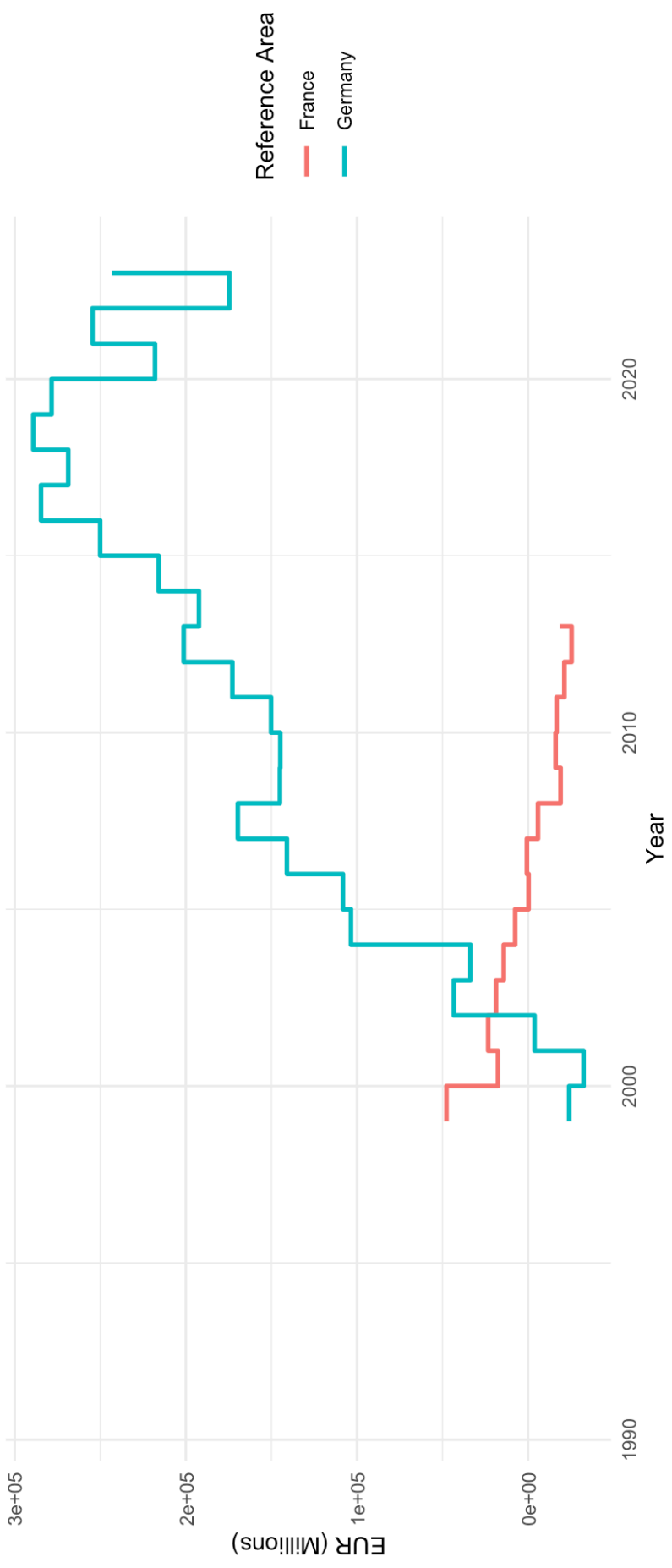
```

1 # Line plot with color by country
2 p2 <- ggplot(bop, aes(x = period, y = value, color = `Reference Area`)) +
3   geom_step(linewidth = 1) +
4   labs(
5     title = "Balance of Payments (BCA, EUR)",
6     subtitle = "France vs Germany – Annual",
7     x = "Year",
8     y = "EUR (Millions)",
9     caption = "Source: IMF / DBnomics"
10  ) +
11    theme_minimal()
12 p2

```

Balance of Payments (BCA, EUR)

France vs Germany — Annual



Source: IMF / DBnomics

Fetch two series from different datasets of different providers

```
1 unemp2 <- rdb(ids = c("AMECO/ZUTN/EA19.1.0.0.0.ZUTN", "Eurostat/une_rt_q/Q.SA.Y15-24.PC_ACT.T.EA19"))
```

```
1 # See which providers and datasets are included
2 dim(unemp2)
```

```
[1] 122 27
```

```
1 unique(unemp2$provider_code)
```

```
[1] "AMECO" "Eurostat"
```

```
1 unique(unemp2$dataset_code)
```

```
[1] "ZUTN" "une_rt_q"
```

```
1 unique(unemp2$series_code)
```

```
[1] "EA19.1.0.0.0.ZUTN" "Q.SA.Y15-24.PC_ACT.T.EA19"
```

```
1 unique(unemp2$`@frequency`)
```

```
[1] "annual" "quarterly"
```

```
1 unique(unemp2$`Seasonal adjustment`)
```

```
[1] NA
```

```
[2] "Seasonally adjusted data, not calendar adjusted data"
```

```

1 # Summarize coverage and data availability
2 unemp2_summary <- unemp2 %>%
3   group_by(series_code) %>%
4   summarize(
5     provider = first(provider_code),
6     dataset = first(dataset_code),
7     start_all = min(period, na.rm = TRUE),
8     end_all = max(period, na.rm = TRUE),
9     start_data = min(period[!is.na(value)]),
10    end_data = max(period[!is.na(value)]),
11    n_obs = sum(!is.na(value)),
12    .groups = "drop"
13  )

```

```

1 unemp2_summary_table <- unemp2_summary |>
2 gt() %>%
3   tab_header(
4     title = "Time Coverage and Non-Missing Observations",
5     subtitle = "For Each Series from AMECO and Eurostat"
6   ) %>%
7   cols_label(
8     series_code = "Series ID",
9     provider = "Provider",
10    dataset = "Dataset",
11    start_all = "Start (all)",
12    end_all = "End (all)",
13    start_data = "Start (non-NA)",
14    end_data = "End (non-NA)",
15    n_obs = "# Obs"
16  ) %>%
17  fmt_date(
18    columns = c(start_all, end_all, start_data, end_data),
19    date_style = "iso"
20  ) %>%
21  tab_options(
22    table.width = pct(100),
23    column_labels.font.weight = "bold"
24  )

```


Time Coverage and Non-Missing Observations							
For Each Series from AMECO and Eurostat							
Series ID	Provider	Dataset	Start (all)	End (all)	Start (non-NA)	End (non-NA)	# Obs
EA19.1.0.0.0.ZUTN	AMECO	ZUTN	1960-01-01	2026-01-01	1997-01-01	2026-01-01	30
Q.SA.Y15-24.PC_ACT.T.EA19	Eurostat	une_rt_q	2009-01-01	2022-07-01	2009-01-01	2022-07-01	55

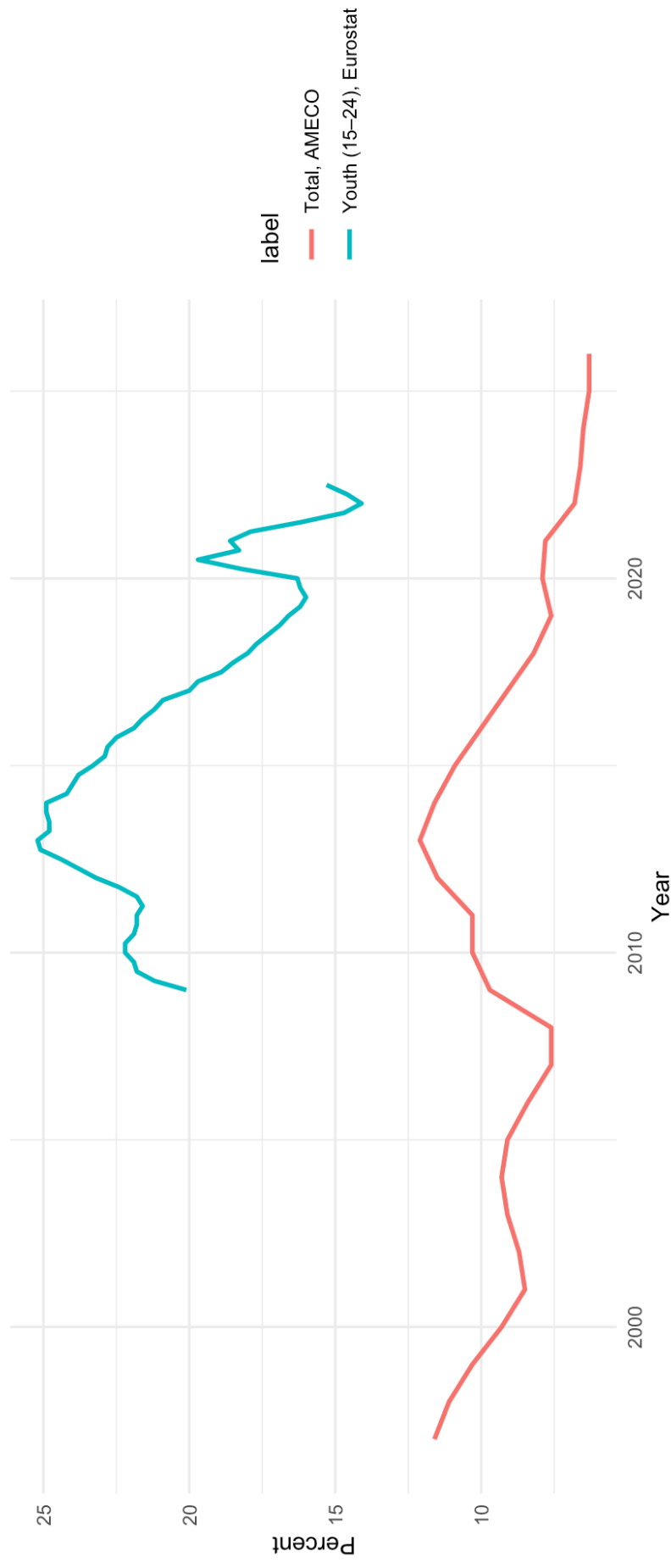
```
1 # Metadata vectors
2 providers <- unique(unemp2$provider_code)
3 datasets <- unique(unemp2$dataset_code)
4 series_ids <- unique(unemp2$series_code)
```

```
1 # Create a label that combines dataset + series ID
2 unemp2_clean <- unemp2 %>%
3   drop_na(value) %>%
4   mutate(label = case_when(
5     series_code == "EA19.1.0.0.0.ZUTN" ~ "Total, AMECO",
6     series_code == "Q.SA.Y15-24.PC_ACT.T.EA19" ~ "Youth (15-24), Eurostat",
7     TRUE ~ series_code
8   ))
```

```
1 p3 <- ggplot(unemp2_clean, aes(x = period, y = value, color = label)) +
2   geom_line(linewidth = 1) +
3   labs(
4     title = "Unemployment Rates from Multiple Sources (EA19)",
5     subtitle = "AMECO and Eurostat – Different definitions",
6     x = "Year", y = "Percent",
7     caption = paste("Series IDs:", paste(unique(unemp2_clean$series_code), collapse = " | "))
8   ) +
9   theme_minimal()
10 p3
```

Unemployment Rates from Multiple Sources (EA19)

AMECO and Eurostat — Different definitions



Series IDs: EA19.1.0.0.0.ZUTN | Q.SA.Y15-24.PC_ACT.T.EA19

Fetch large amounts of data

```
1 cci_ee <- rdb("OECD", "DSD_HHDASH@DF_HHDASH_CTRY", "Q.EST.CCICP.IX")
2 unique(cci_ee$series_name)
```

```
[1] "Quarterly – Estonia – Consumer confidence – Index"
```

```
1 unique(cci_ee$REF_AREA)
```

```
[1] "EST"
```

Wildcarding dimensions

- To fetch **multiple values** for a dimension (e.g. countries), just **remove** the value from that position
 - Example: remove **"EE"** to fetch all countries (**REF_AREA**)

⚠ This can take a while

```
1 # cci_ee <- rdb("OECD", "DSD_HHDASH@DF_HHDASH_CTRY", "Q.EST.CCICP.IX")
2 cci <- rdb("OECD", "DSD_HHDASH@DF_HHDASH_CTRY", "Q..CCICP.IX")
3 # cci <- rdb("OECD", "DSD_HHDASH@DF_HHDASH_CTRY", "..CCICP.IX") # also wildcard the frequency
4 unique(cci$REF_AREA)
```

```
[1] "AUS"      "AUT"      "BEL"      "CHE"      "CHL"      "COL"
[7] "CRI"      "CZE"      "DEU"      "DNK"      "EA"       "ESP"
[13] "EST"      "EU27_2020" "FIN"      "FRA"      "G7"       "GBR"
[19] "GRC"      "HUN"      "IRL"      "ISR"      "ITA"      "JPN"
[25] "KOR"      "LTU"      "LUX"      "LVA"      "MEX"      "NLD"
```

```
[31] "NZL" "OECD" "POL" "PRT" "SVK" "SVN"
[37] "SWE" "TUR" "USA"
```

```
1 cci_eu <- cci %>%
2   filter(
3     REF_AREA %in% c("EU27_2020", "AUT", "EST", "FIN", "DEU", "LVA", "LTU", "POL", "SWE")) |>
4   mutate(
5     REF_AREA = case_when(
6       REF_AREA == "EU27_2020" ~ "EU27",
7       TRUE ~ countrycode(REF_AREA, origin = "iso3c", destination = "iso2c")
8     )
9   )
10  unique(cci_eu$`Reference area`)
```

```
[1] "Austria"
[2] "Germany"
[3] "Estonia"
[4] "European Union (27 countries from 01/02/2020)"
[5] "Finland"
[6] "Lithuania"
[7] "Latvia"
[8] "Poland"
[9] "Sweden"
```

```
1 library(gghighlight)
2 desired_order <- c(
3   "European Union (27 countries from 01/02/2020)",
4   "Austria",
5   "Germany",
6   "Estonia",
7   "Finland",
8   "Lithuania",
9   "Latvia",
10  "Poland",
11  "Sweden"
12 )
13 unique(cci_eu$`Reference area`)
```

```

[1] "Austria"
[2] "Germany"
[3] "Estonia"
[4] "European Union (27 countries from 01/02/2020)"
[5] "Finland"
[6] "Lithuania"
[7] "Latvia"
[8] "Poland"
[9] "Sweden"

```

```

1 cci_eu |>
2   ggplot() +
3     geom_hline(yintercept = 100, linetype = "solid", size = .25) +
4     geom_point(
5       data = cci_eu %>%
6         group_by(`Reference area`) %>%
7         slice_max(period),
8       aes(x = period, y = value, color = `Reference area`),
9       shape = 16
10    ) +
11    geom_line(aes(x = period, y = value, color = `Reference area`)) +
12    gghighlight(
13      use_direct_label = FALSE,
14      unhighlighted_params = list(colour = alpha("grey85", 1))
15    ) +
16    geom_text(
17      data = cci_eu %>%
18        group_by(`Reference area`) %>%
19        slice_max(period),
20      aes(x = period, y = value, color = `Reference area`, label = round(value)),
21      hjust = -.5,
22      vjust = .5,
23      size = 2.5,
24      fontface = "bold"
25    ) +

```

- Sometimes you need to fetch many if not all dimensions of the data
- You can wildcard dimension and post-filter
- Example: MFI Interest Rate Statistics from the ECB
 - Start with a [single series](#) (Estonia, mortgage rates)

```
1 mir_mortgage_ee <- rdb("ECB", "MIR", "M.EE.B.A2C.A.R.A.2250.EUR.N")
2 unique(mir_mortgage_ee$series_name)
```

```
[1] "Monthly – Estonia – Deposit-taking corporations except the central bank (S.122) – Lending for house  
purchase excluding revolving loans and overdrafts, convenience and extended credit card debt – Total –  
Annualised agreed rate (AAR) / Narrowly defined effective rate (NDER) – Total – Households and non-profit  
institutions serving households (S.14 and S.15) – Euro – New business"
```


Wildcarding dimensions

- To fetch multiple values for a dimension (e.g. countries), just **remove** the value from that position
 - Example: remove **"EE"** to fetch all countries (**REF_AREA**)

⚠ This can take a while

```
1 # mir_mortgage_ee <- rdb("ECB", "MIR", "M.EE.B.A2C.A.R.A.2250.EUR.N")
2 mir <- rdb("ECB", "MIR", "M..B..A.R.A..EUR.N")
3 unique(mir$REF_AREA)
```

```
[1] "AT" "BE" "CY" "DE" "EE" "ES" "FI" "FR" "GR" "HR" "IE" "IT" "LT" "LU" "LV"
[16] "MT" "NL" "PT" "SI" "SK" "U2"
```

```
1 unique(mir$BS_ITEM)
```

```
[1] "A2A" "A2AC" "A2B" "A2BC" "A2C" "A2CC" "A2D" "A2Z" "A2Z1" "A2Z3"
[11] "L21" "L22" "L23" "L24"
```

```
1 unique(mir$`BS counterpart sector`)
```

```
[1] "Non-Financial corporations (S.11)"
[2] "Households and non-profit institutions serving households (S.14 and S.15)"
[3] "Households of which sole proprietors and unincorporated partnerships (SP/UP)"
[4] "Non-Financial corporations and Households (S.11 and S.14 and S.15)"
```

Filter and plot

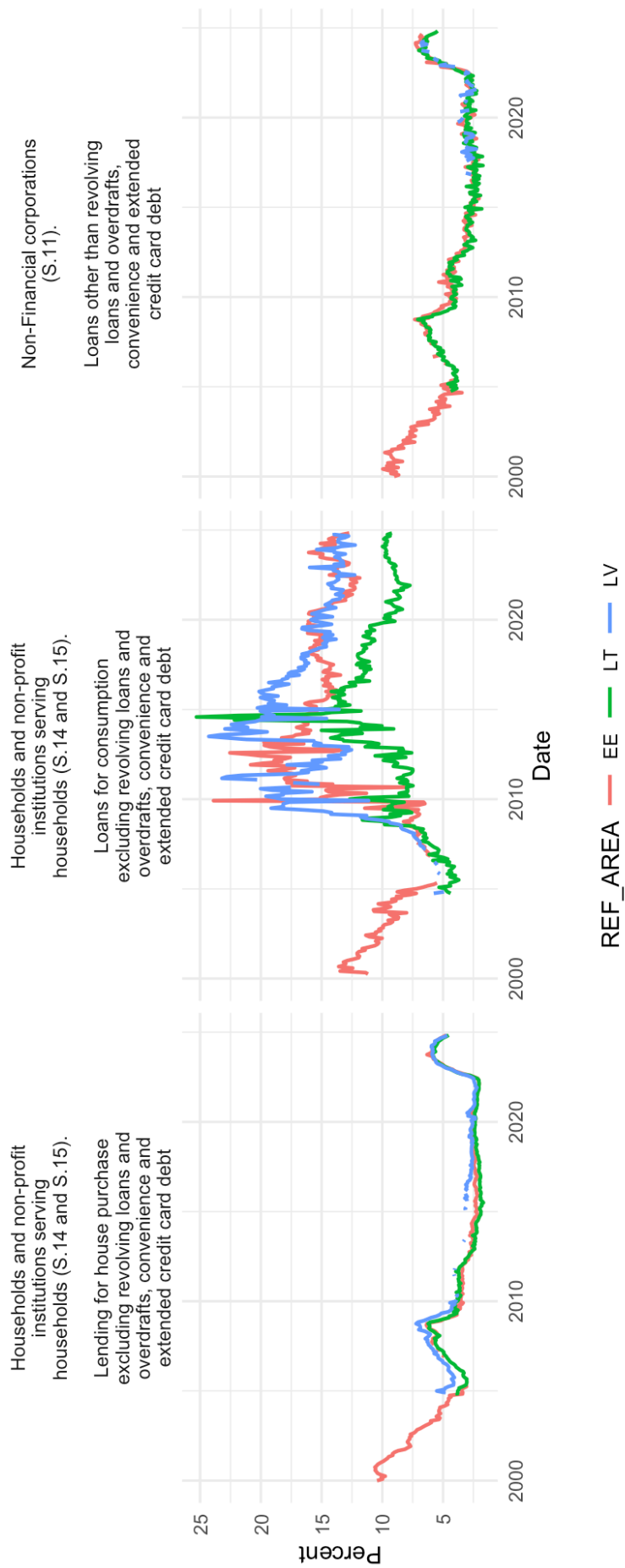
- Filter Estonia, Latvia and Lithuania
- Keep only selected **BS items** (loan categories)

```
1 # Filter by BS_ITEM and countries
2 mir_filtered <- mir %>%
3   filter(
4     REF_AREA %in% c("EE", "LV", "LT"),
5     BS_ITEM %in% c("A2I", "A2C", "A2B", "A2J", "A2A")
6   )
```

Plot interest rates by country & type

```
1 country_list <- paste(sort(unique(mir_filtered$REF_AREA)), collapse = ", ")
2 item_list <- paste(unique(mir_filtered$BS_ITEM), collapse = ", ")
3
4 caption_text <- paste(
5   "Source: ECB / DBnomics – Dataset code: MIR",
6   paste0("\nFiltered: REF_AREA in ", country_list, "; BS_ITEM in ", item_list)
7 )
8
9 mir_filtered <- mir_filtered %>%
10   mutate(facet_label = paste0(`BS counterpart sector`, ".\n\n", `Balance sheet item`))
11
12 p4 <- ggplot(mir_filtered, aes(x = period, y = value, color = REF_AREA)) +
13   geom_line(linewidth = 0.8) +
14   facet_wrap(~ facet_label, labeller = label_wrap_gen(width = 30), ncol = 3) +
15   labs(
16     title = "Interest Rates for Households and Firms",
17     subtitle = "Faceted by Loan Type and Borrower Sector",
18     x = "Date", y = "Percent",
19     caption = caption_text
20   ) +
21   theme_minimal() +
22   theme(legend.position = "bottom")
23 p4
```

Interest Rates for Households and Firms
Faceted by Loan Type and Borrower Sector



Source: ECB / DBnomics — Dataset code: MIR
Filtered: REF_AREA in EE, LT, LV; BS_ITEM in A2A, A2B, A2C

Accessing Statistics Estonia Data with **pxweb**

- **Statistics Estonia** is not yet part of DBnomics
- No dedicated R package for andmed.stat.ee
- But: you can access data using the [pxweb](#) R package
- Based on the **PX-WEB API**, which is also used by many Nordic stats agencies

How to Identify Variables

How to Identify Query Variables

-  Use either:

1. Programmatically via the metadata (`pxweb_get()`, `pxweb_get_metadata()`)
2. From the Statistics Estonia web interface – click “API query for this table”

```
1 pa111_meta <- pxweb_get(  
2   url = "https://andmed.stat.ee/api/v1/en/stat/PA111")  
3 pa111_meta
```

PXWEB METADATA

PA111: AVERAGE MONTHLY GROSS WAGES (SALARIES), MEDIAN, DECILES AND NUMBER OF EMPLOYEES BY ECONOMIC ACTIVITY SECTION (QUARTERLY)

variables:

```
[[1]] Näitaja: Indicator  
[[2]] Tegevusala: Economic activity  
[[3]] Vaatlusperiood: Reference period
```

```
1 pa111_meta$variables[[1]]
```

\$code

```
[1] "Näitaja"
```

\$text

```
[1] "Indicator"
```

\$values

```
[1] "GR_W_AVG"  
[6] "GR_W_D4"  
[11] "GR_W_D9"  
[1] "NR_EMPL"  
[6] "GR_W_D5"  
[11] "GR_W_AVG_SM"  
[1] "GR_W_D2"  
[6] "GR_W_D7"  
[11] "GR_W_D3"  
[6] "GR_W_D8"
```

```

$valueTexts
[1] "Average monthly gross wages (salaries), euros"
[2] "Number of employees"
[3] "1st decile of monthly gross wages (salaries), euros"
[4] "2nd decile of monthly gross wages (salaries), euros"
[5] "3rd decile of monthly gross wages (salaries), euros"
[6] "4th decile of monthly gross wages (salaries), euros"
[7] "Median (5th decile) of monthly gross wages (salaries), euros"

```



```

1 px_query_list1 <- list(
2   "Näitaja" = c("GR_W_D1", "GR_W_D5", "GR_W_D9"),
3   "Tegevusala" = "*",
4   "Vaatlusperiood" = "*"
5 )
6
7 pa111 <- pxweb_get(
8   url = "https://andmed.stat.ee/api/v1/en/stat/PA111",
9   query = px_query_list1
10 )
11
12 # Convert to data frame
13 pa111.df <- as.data.frame(pa111, column.name.type = "text", variable.value.type = "text")
14 pa111.code <- as.data.frame(pa111, column.name.type = "code", variable.value.type = "code")
15 pa111.df$Näitaja <- pa111.code$Näitaja

```

```

1 selected_sectors <- c("Manufacturing", "Construction", "Information and communication", "Real estate activit
2 # Clean and reshape
3 pa111_plotdata <- pa111.df %>%
4   mutate(
5     quarter = yq(str_replace(`Reference period`, " ", "-")),
6     Indicator = recode(
7       Näitaja,
8       "GR_W_D1" = "D1",
9       "GR_W_D5" = "Median",
10      "GR_W_D9" = "D9"
11    )
12   ) %>%
13   filter(
14     `Economic activity` %in% selected_sectors,
15     Indicator %in% c("D1", "Median", "D9")
16   ) %>%
17   select(quarter, `Economic activity`, Indicator, value = `PA111: AVERAGE MONTHLY GROSS WAGES (SALARIES)`, ME
18   pivot_wider(names_from = Indicator, values_from = value)

```

```
1 overall_median <- pa111_plotdata %>%  
2   filter(`Economic activity` == "Total - all activities") %>%  
3   select(quarter, total_median = Median)
```

```
1 pa111_plotdata <- pa111_plotdata %>%  
2   filter(`Economic activity` != "Total - all activities") %>%  
3   left_join(overall_median, by = "quarter")
```

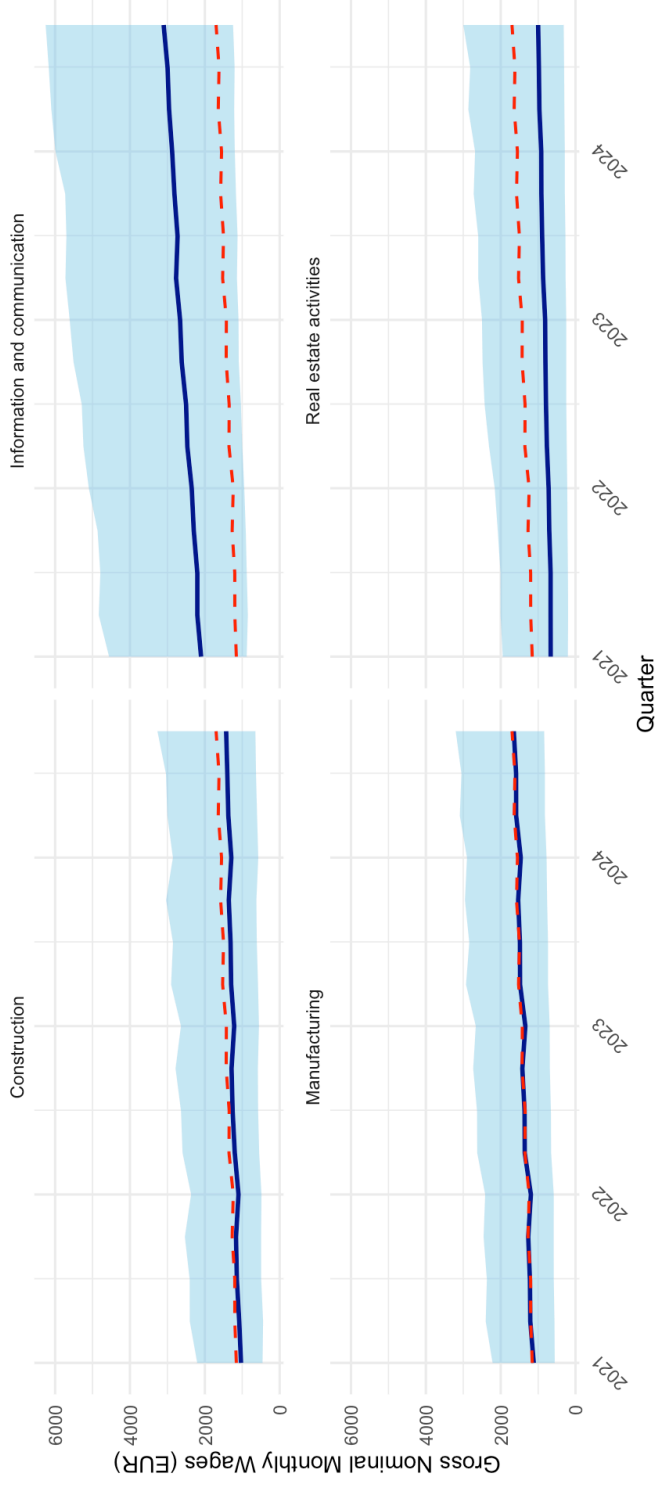
```

1 ggplot(pa111_plotdata, aes(x = quarter)) +
2   geom_ribbon(aes(ymin = D1, ymax = D9), fill = "skyblue", alpha = 0.5) +
3   geom_line(aes(y = Median), color = "darkblue", linewidth = 1) +
4   geom_line(aes(y = total_median), color = "red", linetype = "dashed", linewidth = 0.7) +
5   facet_wrap(~`Economic activity`, ncol = 2) +
6   labs(
7     title = "Wage Distribution by Sector (1st to 9th Decile, Median)",
8     subtitle = "Red dashed line shows overall median (Total - all activities)",
9     x = "Quarter",
10    y = "Gross Nominal Monthly Wages (EUR)"
11  ) +
12  theme_minimal() +
13  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

Wage Distribution by Sector (1st to 9th Decile, Median)

Red dashed line shows overall median (Total - all activities)



```
1 # library(DataExplorer)
2 # create_report(pa111_plotdata)
```